

# Actor-Based Distributed Systems

Marek Konieczny

[marekko@agh.edu.pl](mailto:marekko@agh.edu.pl),

Room 4.43, Spring 2026



AKADEMIA GÓRNICZO-HUTNICZA  
IM. STANISŁAWA STASZICA  
W KRAKOWIE

# Class Logistics

- If you want to use your laptop, use the attached docker-compose.yaml file
- We will use an external cluster and a Jupyter notebook file attached in the labs

# Class Logistics

- Laboratory classes during week 18-22.4
- You can use desktops from the Computer Networks Laboratory
  - Select the Ubuntu image during boot
- Preliminary plan homework assignment
  - 1-3 10-11 June

# Class Logistics

- Laboratory grading
  - Showing up => 1 pt
  - UPEL task upload => 1 pt
  - Hard work on lab meeting => 2 pt
    - You need to show logs of your tasks execution on Ray Dashboard, and on the AWS console
  - Extra activity, labs improvement => 1 pt

# Actors (theory)

- Mathematical model of concurrent computation proposed by Carl Hewitt in 1973

Session 8 Formalisms for  
Artificial Intelligence

## A Universal Modular ACTOR Formalism for Artificial Intelligence

Carl Hewitt

Peter Bishop

Richard Steiger

### Abstract

This paper proposes a modular ACTOR architecture and definitional method for artificial intelligence that is conceptually based on a single kind of object: actors [or, if you will, virtual processors, activation frames, or streams]. The formalism makes no presuppositions about the representation of primitive data structures and control structures. Such structures can be programmed, micro-coded, or hard wired in a uniform modular fashion. In fact it is impossible to determine whether a given object is "really" represented as a list, a vector, a hash table, a function, or a process. The architecture will efficiently run the coming generation of PLANNER-like artificial intelligence languages including those requiring a high degree of parallelism. The efficiency is gained without loss of programming generality because it only makes certain actors more efficient; it does not change their behavioral characteristics. The architecture is general with respect to control structure and does not have or need goto, interrupt, or semaphore primitives. The formalism achieves the goals that the disallowed constructs are intended to achieve by other more structured methods.

PLANNER Progress

# Actors (theory)

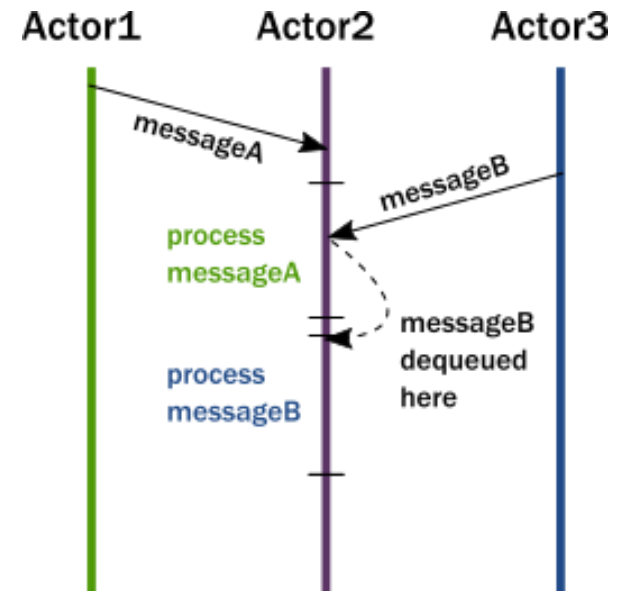
- Primitives for concurrency/parallelism
- Entities having a message queue and associated behavior → **And isolated state!!**
- Can exchange messages between each-other
- When an actor receives a message it can:
  - Send a finite number of messages to other actors
  - Create a finite number of new actors
  - Modify its internal behavior on receiving messages

# Actors (theory)

- Messages between actors are always sent asynchronously
- No requirement on the order of message arrival
- Queuing and dequeuing of messages in an actor mailbox are atomic operations
- Fire and Forget approach, **no race conditions anymore**

# Actors (theory)

- The actor is an object that encapsulates state and behavior
- Originated from
  - Smalltalk, Petri Nets, Channels ...
  - More details on theory in attached document
- More recent
  - *Scheme* made them concrete
  - *Erlang* made them useful
  - *Akka* made them cool
  - *Ray* makes them easy





# Origins

- Originated on UC Berkeley
- Group led by Ion Stoica:
  - Apache Messos, Apache Spark (Databricks), Apache Alluxio (Alluxio)
  - Ray is managed by Anyscale



## **Ray: A Distributed Framework for Emerging AI Applications**

Philipp Moritz\*, Robert Nishihara\*, Stephanie Wang, Alexey Tumanov, Richard Liaw,  
Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, Ion Stoica  
*University of California, Berkeley*

# Origins

## PARK Stack

PyTorch · AI · Ray · Kubernetes

 PyTorch



 RAY

 kubernetes

**10-100x**

more model training data.

 instacart



**1M+**

CPU cores deployed for online model serving.

 ANT GROUP



**300B+**

parameters for foundation model training.

 OpenAI



**82%**

lower data processing cost (\$120M savings per year)

 amazon



**30x**

cost reduction switching from Spark to Ray for batch inference processing with GPUs

 Pinterest



**4x**

improvement in GPU utilization and 7X lower costs

ebay



**10x**

more models trained per month

 reddit



**12x**

faster iteration to deliver over 100+ production models

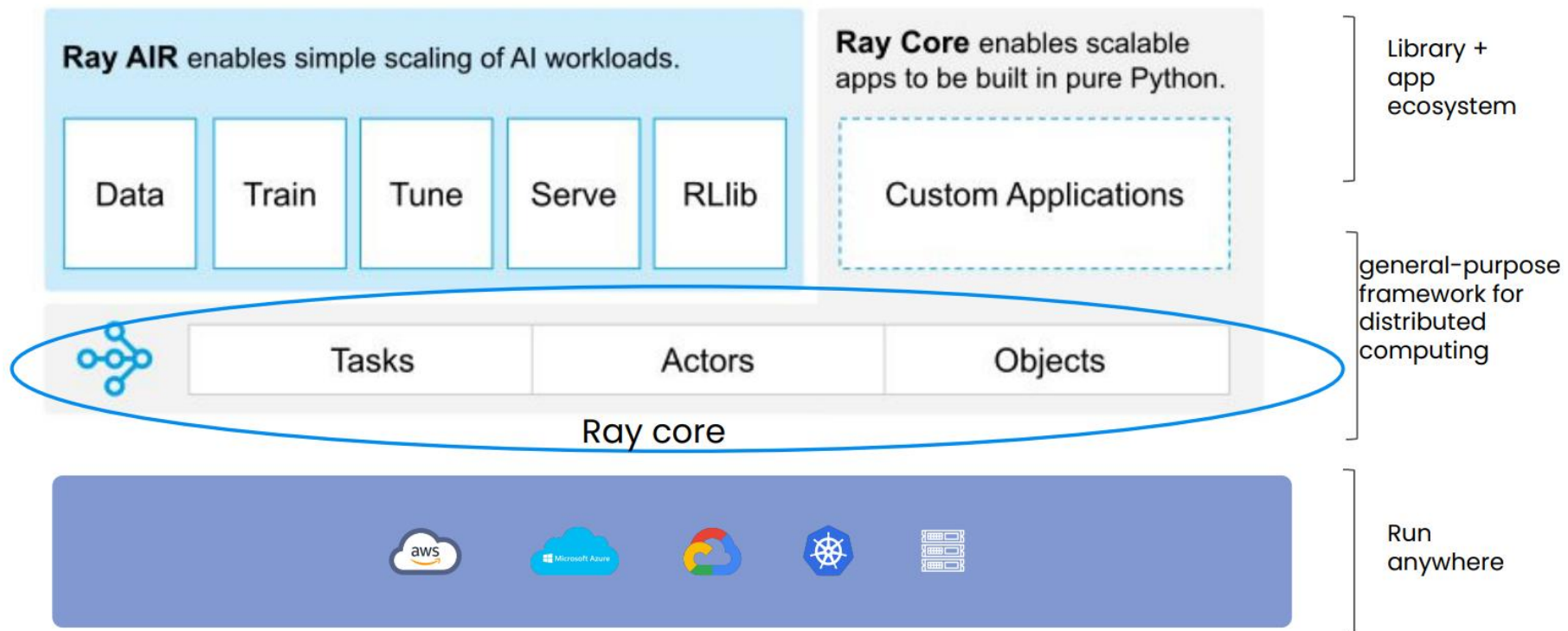
Canva



# The FAST Compute Model

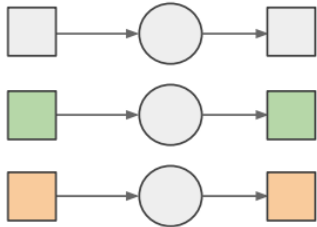
- Most important features
  - **Futures**: references to objects
  - **Actors**: remote class instances (objects)
  - **Shared** in-memory distributed object store
  - **Tasks**: remote functions

# What is Ray ?



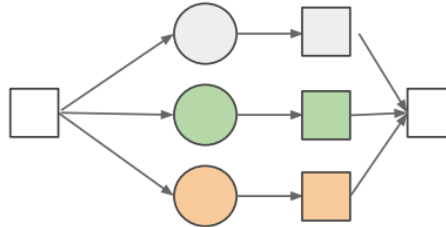
# Scaling patterns

**Batch Training / Inference**



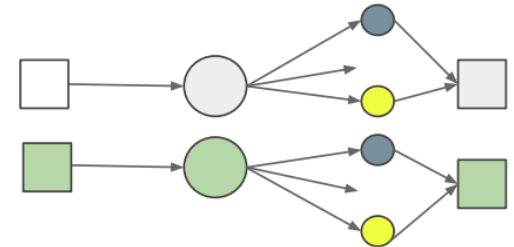
**Different data / Same function**

**AutoML**

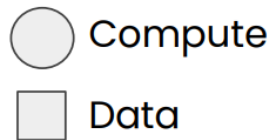


**Same data / Different function**

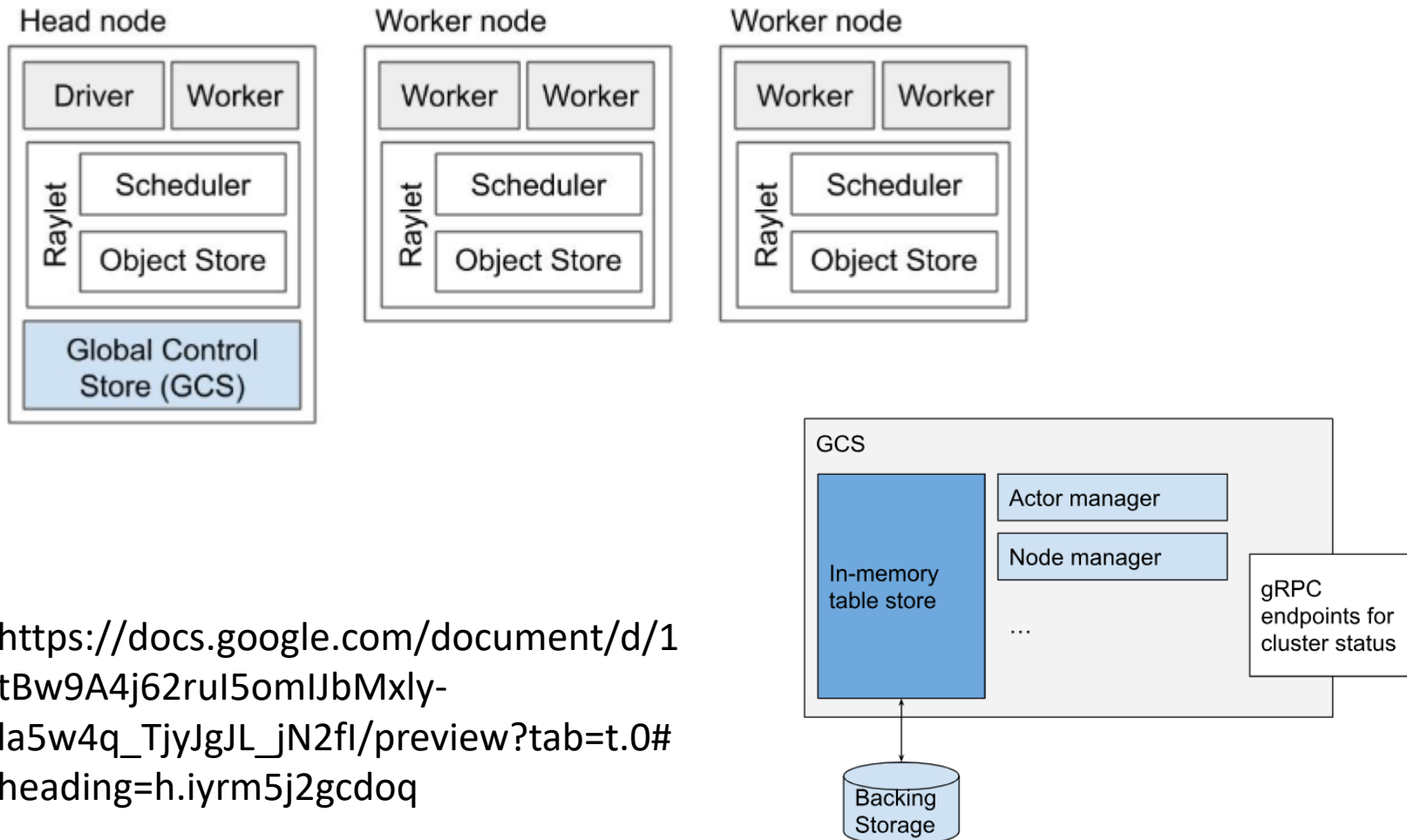
**Batch Tuning**



**Different data / Same function**



# What is Ray ?



[https://docs.google.com/document/d/1tBw9A4j62rul5omIJbMxly-la5w4q\\_TjyJgJL\\_jN2fl/preview?tab=t.0#heading=h.iyrm5j2gcdoq](https://docs.google.com/document/d/1tBw9A4j62rul5omIJbMxly-la5w4q_TjyJgJL_jN2fl/preview?tab=t.0#heading=h.iyrm5j2gcdoq)

# What is Ray ?

Python → Ray APIs



```
def f(x):  
    # do something with x:  
    y= ...  
    return y
```

Task



```
@ray.remote  
def f(x):  
    # do something with x:  
    y= ...  
    return y
```

Distributed



...



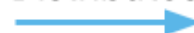
```
class Cls():  
    def __init__(self,  
x):  
    def f(self, a):  
        ...  
    def g(self, a):  
        ...
```

Actor

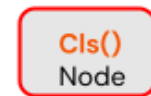


```
@ray.remote  
class Cls():  
    def  
__init__(self, x):  
    def f(self, a):  
        ...  
    def g(self, a):  
        ...
```

Distributed



...



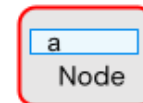
```
import numpy as np  
a= np.arange(1, 10e6)  
b = a * 2
```

Distributed  
immutable  
object

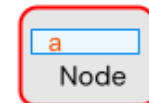


```
import numpy as np  
a = np.arange(1, 10e6)  
obj_a = ray.put(a)  
b = ray.get(obj_a) * 2
```

Distributed



...



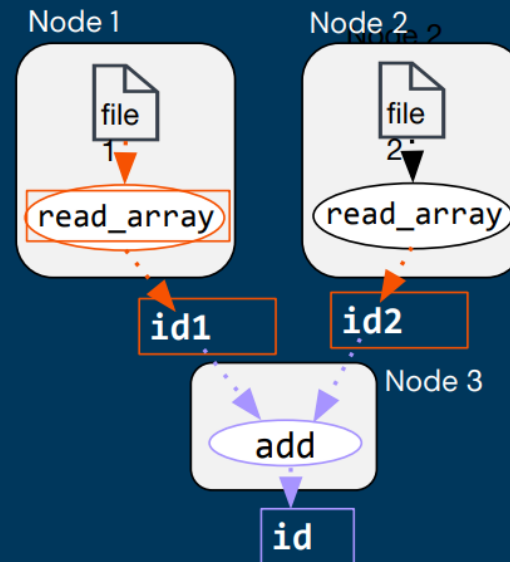
# Task API

```
@ray.remote
def read_array(file):
    # read ndarray "a"
    # from "file"
    return a
```

```
@ray.remote
def add(a, b):
    return np.add(a, b)
```

```
id1 = read_array.remote(file1)
id2 = read_array.remote(file2)
id = add.remote(id1, id2)
sum = ray.get(id)
```

`ray.get()` block until  
result available





# Laboratories

- Start Docker Compose with Jupyter Notebook
  - `http://localhost:8888/?token=raylab`
- Connect to the cluster provided during laboratories
- Read the document and implement assignments

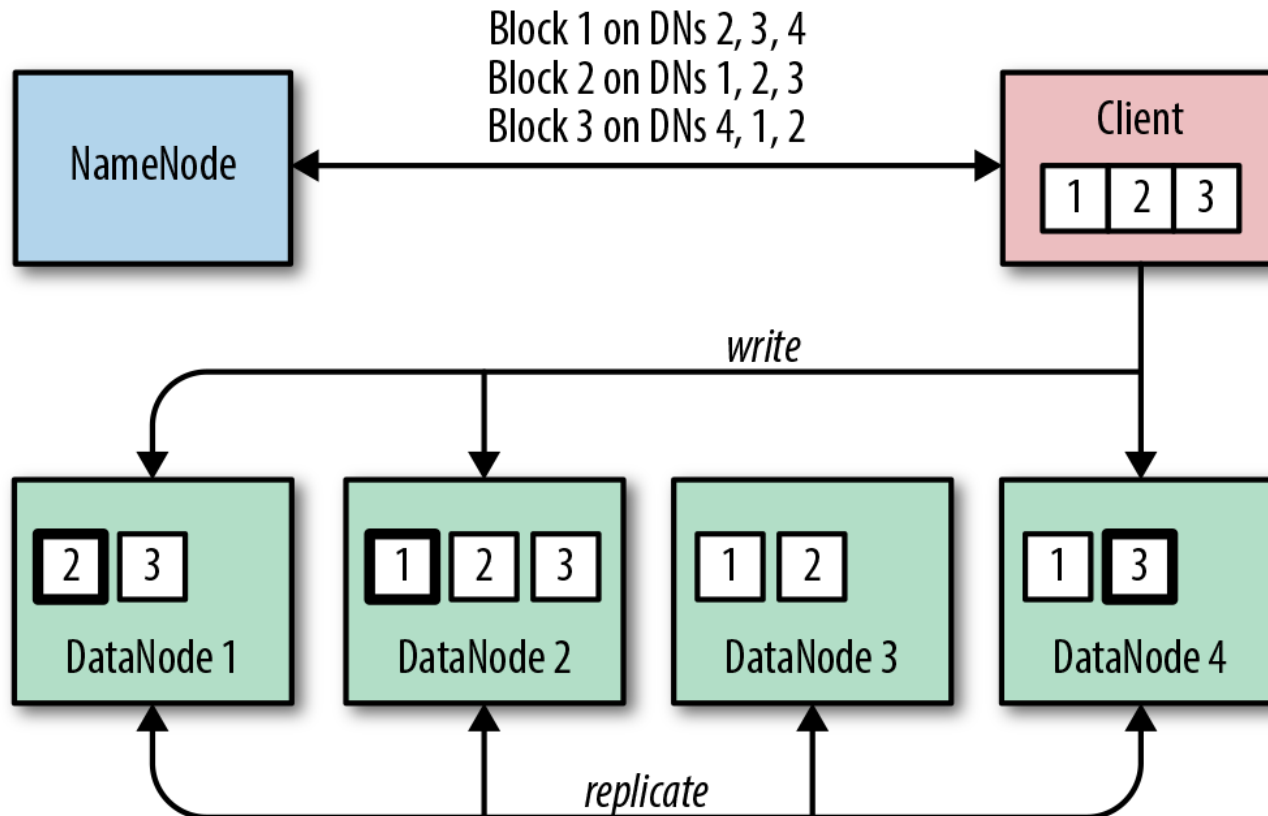
# Homework

- Tier 1 (3 pts)
  - Complete and present all exercises from Laboratories  
=> 3 pts
- Tier 2 (6-9 pts)
  - Implement the Homework and Present demo and test scenarios => 6 pts
  - Add some additional advanced functionality, select anything from items not discussed on labs => 7-9 pts
  - Add distribution of environment (e.g. execute it on a cluster based on K8s, docker ...) => 10 pts

# Homework

- Tier 3 (9 pts)
  - Prepare/migrate sample ML project with other Ray AI Runtime modules, use 3 different modules
  - Please use unique data sets:
    - No MIST, Iris ...
  - Prepare tests for your solution or validation
  - Describe how you move it from regular implementation to demo on top of Ray
- Execute them on top of distributed environment (+ 1 pt) => 10 pts
  - Show console with different nodes available in the cluster

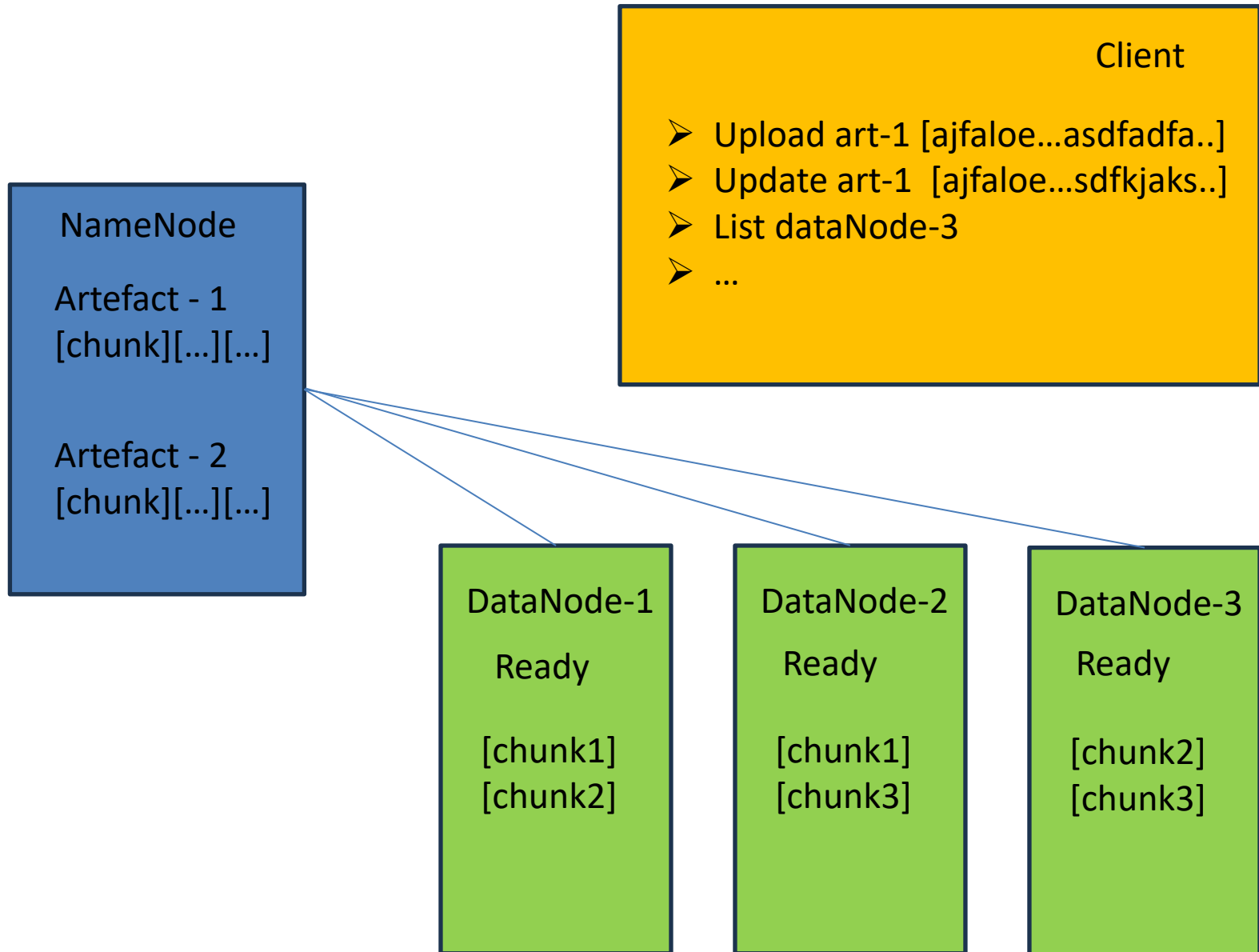
# Homework - Tier 2



# Homework - Tier 2

- Implement a Distributed Artifacts Storage system:
  - There is one name node and  $x$  storage nodes
    - all implemented as actors
  - name node distributes artifacts to storage nodes in chunks/blocks and keeps track of them
  - there are few copies of chunks in the system to improve high availability and speed (similarly as in HDFS )
  - randomly, data storage nodes can fail; the name node keeps track of all chunks and manages the copies
  - In case of failure all data should be consistent – e.g. all copies should synchronize
  - provide a list operation of the most important data (e.g. where chunks/blocks are stored)

# Homework - Tier 2



# Homework - Tier 2

- Artifact consists of name and content
  - stored as a very long string
- The user can perform the following operations
  - upload - by providing a new artifact
  - update - the stored artifact content
  - delete artifacts present in the storage
  - get/download the artifacts from the storage
  - list the status of the cluster
    - To verify the current copies
- For demo purposes, add also node failure cases
- Prepare code for executing the demo before classes